

Języki skryptowe zajęcia-3

Zad1. Narysuj wykres funkcji:

$$f(x, y) = \left\{ \sin 2x + \frac{3}{2} \cos^4 y + 2 \cos (x + y) \right.$$

W obszarze $\left[-\frac{\pi}{2}, \frac{\pi}{2}\right] \times \left[-\frac{\pi}{2}, \frac{\pi}{2}\right]$. Użyj funkcji `contour`, `contourf`, `imshow` i porównaj otrzymane wykresy.

Zad2. Narysuj torus zadany równaniami parametrycznymi:

$$\begin{cases} x = (100 + 60 \cos \beta) \cos \alpha \\ y = (100 + 60 \cos \beta) \sin \alpha \\ z = 60 \sin \beta \end{cases}$$

Rozwiązanie

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from mpl_toolkits.mplot3d import Axes3D
```

```
alfa=np.linspace(0,6.28,51)
```

```
beta=np.linspace(0,6.28,51)
```

```
alfa,beta=np.meshgrid(alfa,beta)
```

```
X=(100+60*np.cos(beta))*np.cos(alfa)
```

```
Y=(100+60*np.cos(beta))*np.sin(alfa)
```

```
Z=60* np.sin(beta)
```

```
fig = plt.figure()
```

```
ax = Axes3D(fig)
```

```
surf=ax.plot_surface(X, Y, Z, rstride=4, cstride=1, linewidth=0.1, alpha=0.5)
ax.auto_scale_xyz([-160, 160], [-160, 160], [-120, 120])
plt.show()
```

Zad3 Narysuj wstęgę Möbiusa zadaną równaniami parametrycznymi:

$$\begin{cases} x = \left(100 + 60\cos\frac{\alpha}{2}t\right)\cos\alpha \\ y = \left(100 + 60\cos\frac{\alpha}{2}t\right)\sin\alpha \\ z = 60\sin\frac{\alpha}{2}t \end{cases}$$

Rozwiązanie

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

alfa,t=np.meshgrid(np.linspace(0,6.28,51),np.linspace(-1,1,51))

X=(100+60*np.cos(alfa/2)*t)*np.cos(alfa)
Y=(100+60*np.cos(alfa/2)*t)*np.sin(alfa)
Z=60* np.sin(alfa/2)*t

fig = plt.figure()
ax = Axes3D(fig)
surf=ax.plot_surface(X, Y, Z, rstride=4, cstride=1, linewidth=0.1, alpha=0.5)
ax.auto_scale_xyz([-160, 160], [-160, 160], [-120, 120])
plt.show()
```

Zad4 Policzmy gradient funkcji f z zadania 1 $f(x, y) = \left\{ \sin 2x + \frac{3}{2} \cos^4 y + 2 \cos(x + y) \right\}$.

$\nabla f = (2 \cos 2x - 2 \sin(x + y), -6 \cos^3 y \sin y - 2 \sin(x + y))$. Na wykres konturowy funkcji f nałóż wykres jej gradientu.

Uwaga!!!! $\nabla f = (2 \cos 2x - 2 \sin(x + y), -6 \cos^3 y \sin y - 2 \sin(x + y))$ -jest to pole wektorowe które w każdym punkcie dziedziny funkcji pokazuje kierunek najszybszego wzrostu funkcji a długość wektora gradientu jest proporcjonalna do stromości wykresu funkcji. Pole wektorowe rysuje funkcja quiver z modułu pyplot. Wywołanie quiver(X,Y,VX,VY). Jej argumenty to tablice 2-wymiarowe równych wymiarów

Rozwiązanie

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

X,Y=np.meshgrid(np.linspace(-1.57,1.57,100),np.linspace(-1.57,1.57,100))
Z=np.sin(2*X)+1.5*np.cos(Y)**4+2*np.cos(X+Y)
plt.contour(X,Y,Z,50)
X=X[::5,::5]
Y=Y[::5,::5]
VX=2*np.cos(2*X)-2*np.sin(X+Y)
VY=-6*np.cos(Y)**3*np.sin(Y)-2*np.sin(X+Y)
plt.quiver(X,Y,VX,VY)
plt.show()
```

Wykresy animowane (Animacja).

Moduł `matplotlib.animation` dostarcza klasę `FuncAnimation`. Jej konstruktor przyjmuje następujące argumenty:

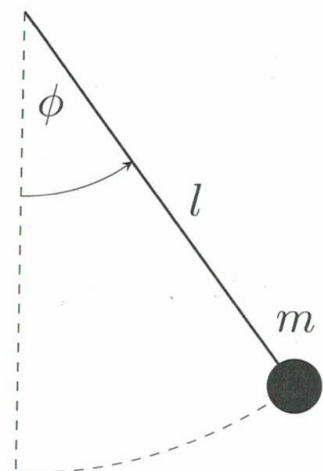
- `fig` — obiekt figury, którego elementy będą się zmieniać w trakcie animacji.
- `func` — funkcja animująca. Przyjmuje jeden argument — chwilę czasu — i nic nie zwraca. Modyfikuje elementy wykresu poprzez zmienne globalne.
- `frames` — zakres czasu.
- `init_func` — funkcja inicjująca, ustawia początkowy stan figury. Nie przyjmuje żadnych argumentów i modyfikuje argumenty wykresu poprzez zmienne globalne.
- `interval` — czas trwania jednej klatki w milisekundach, domyślnie 200.
- `repeat` — określa, czy animacja ma się powtarzać po zakończeniu, domyślnie `True`.

Zad5 Rozwiązać równanie różniczkowe wahadła matematycznego. Znaleźć krzywą fazową. Krzywa fazowa jest styczna do pola wektorowego w każdym punkcie pola.

Rozważmy wahadło matematyczne — punktową masę na nieważkim sztywnym pręcie zaczepionym trwale w drugim końcu. Jego ruch odbywa się w pewnej płaszczyźnie, a kąt ϕ odchylenia od osi pionowej spełnia równanie różniczkowe wynikające z drugiej zasady dynamiki dla ruchu obrotowego:

$$ml^2\ddot{\phi} = -mgl \sin \alpha \quad \Rightarrow \quad \ddot{\phi} = -\omega_0^2 \sin \alpha$$

gdzie $\omega_0 = \sqrt{\frac{g}{l}}$.



Kropka nad zmienną oznacza pochodną po czasie. Równanie takie należy zawsze przekształcić do układu równań pierwszego rzędu. Najprościej robimy to, definiując jako dodatkowe zmienne prędkości — pochodne zmiennych występujących w równaniu:

$$\begin{cases} \dot{\phi} = \omega \\ \dot{\omega} = -\omega_0^2 \sin \phi \end{cases}$$

W każdym punkcie (ϕ, ω) mamy pewien wektor pochodnej czasowej, a zatem zdefiniowaliśmy pewne pole wektorowe na przestrzeni fazowej układu $[-\frac{\pi}{2}, \frac{\pi}{2}] \times \mathbb{R}$. Rozwiązanie równania różniczkowego polega na znalezieniu krzywej w każdym punkcie stycznej do tego pola wektorowego (krzywej fazowej).

- Pole wektorowe — jest to funkcja przyjmująca dwa argumenty: punkt przestrzeni fazowej i chwilę czasu zwracającą wektor pochodnej w tym punkcie. Możliwość uzależnienia funkcji od czasu pozwala na rozwiązywanie układów równań nieautonomicznych, w których parametry zmieniają się w czasie. W przypadku równań autonomicznych definiujemy domyślną wartość drugiego argumentu, żeby nie trzeba było go podawać w sposób jawny.
- Punkt początkowy.
- Zakres czasu — tablica numpy.

Rozwiązaniem funkcji jest tablica, której kolejnymi kolumnami są przebiegi czasowe kolejnych zmiennych stanu. Obliczmy odcinek pierwszych 15 sekund trajektorii fazowej startującej z punktu $[3, 0]$ i narysujmy ją na tle pola wektorowego (dla $\omega_0 = 1$, co odpowiada długości wahadła 9.81 m):

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

def pole(punkt, t=0):
    return (punkt[1], -np.sin(punkt[0]))

fi=np.linspace(-3.14,3.14,21)
om=np.linspace(-2,2,21)
fi,om=np.meshgrid(fi,om)

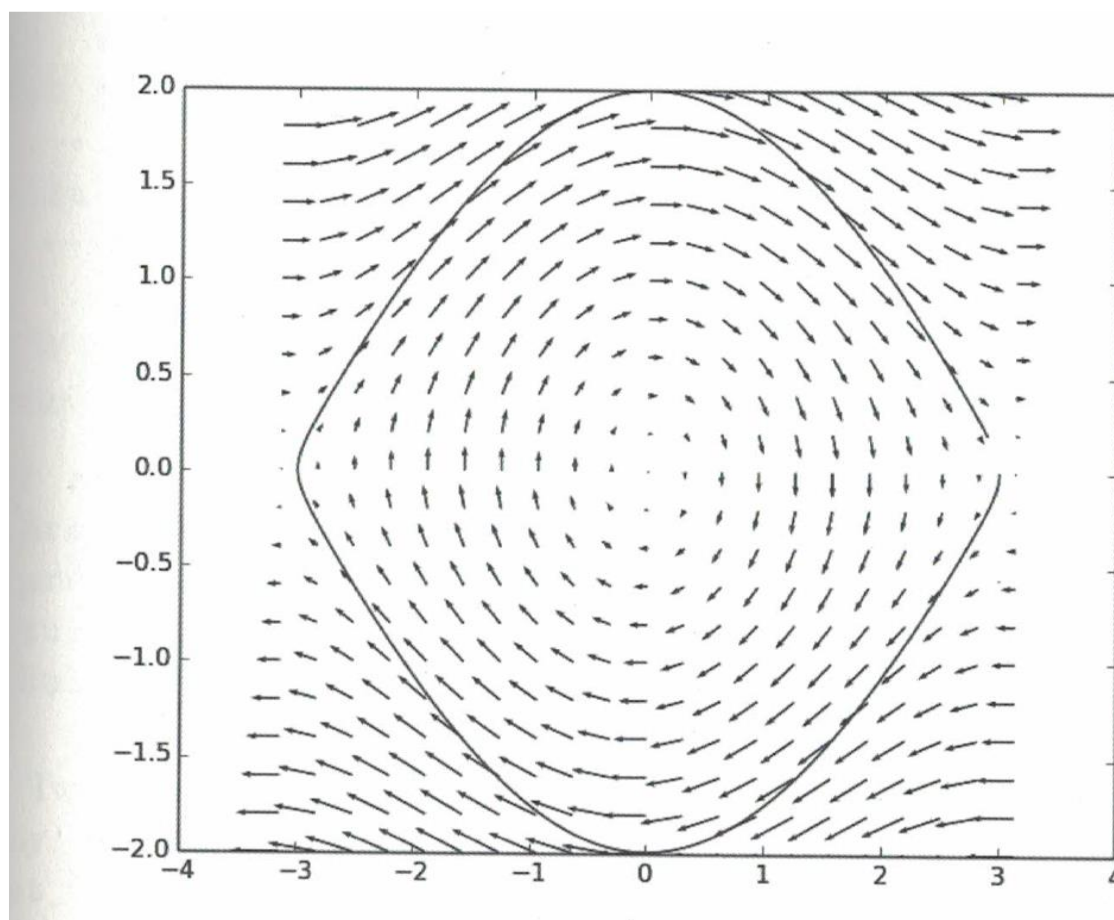
dfi,dom=pole((fi,om))
plt.quiver(fi,om,dfi,dom,width=0.002,scale=40)
t=np.linspace(0,15,1501)
rozw=odeint(pole,[3,0],t)
plt.plot(rozw[:,0],rozw[:,1])
plt.show()
```

Po wykonaniu niezbędnych importów definiujemy funkcję `pole` przypisującą punktowi w przestrzeni fazowej wartość pochodnej (funkcję tę określa układ równań 8.1). Zgodnie ze specyfikacją funkcji `odeint` funkcja `pole` (która będzie jej argumentem) musi przyjmować jako drugi argument czas, nawet jeżeli nie będzie on nigdzie użyty. Żeby nie musieć go podawać, ustawiamy mu domyślną wartość $t=0$.

Następnie definiujemy dwuwymiarową siatkę obliczeniową za pomocą funkcji `meshgrid` i generujemy składowe pola wektorowego `dfi`, `dom`. Ponieważ użyliśmy funkcji `numpy.sin`, która działa na tablicach punktowo, funkcja `pole` potrafi również poprawnie zadziałać na całych tablicach punktów. Wykorzystując te tablice, rysujemy pole wektorowe.

Wywołujemy funkcję `odeint`, podając jej za argumenty funkcję opisującą pole wektorowe, punkt początkowy i siatkę czasu. W pierwszej kolumnie tablicy `rozv` znajdują się składowe ϕ punktów trajektorii, a w drugiej kolumnie — składowe ω . Rysujemy trajektorię, podając pierwszą i drugą kolumnę tablicy `rozv` jako argumenty funkcji `plot`.

Rozwiązanie



Zad6-105 Dodaj do pola wektorowego wahadła rodzinę trajektorii startujących z punktów $[\phi_0, 0]$ gdzie ϕ_0 przyjmuje wartości $\{0.1, 0.3, \dots, 3.1\}$.

Rozwiązanie.

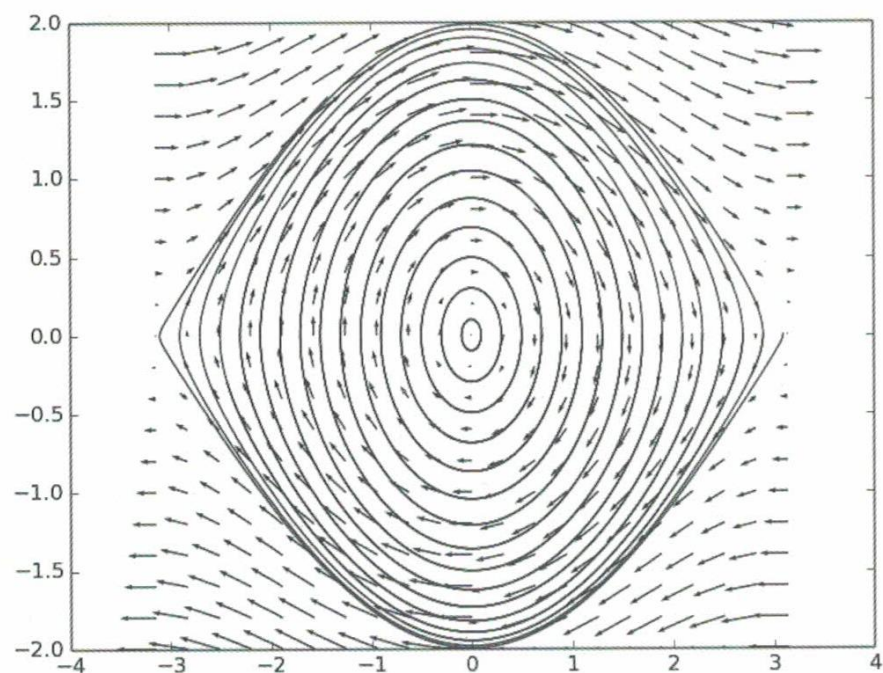
W programie rysującym portret fazowy zamieniamy linie:

```
rozv = odeint(pole, [3, 0], t)
plt.plot(rozv[:, 0], rozv[:, 1])

na

for phi0 in np.linspace(.1, 3.1, 16):
    rozv = odeint(pole, [phi0, 0], t)
    plt.plot(rozv[:, 0], rozv[:, 1], 'b')
```

W wyniku otrzymujemy wykres:



Rozwiązanie Zad6-105

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint
```

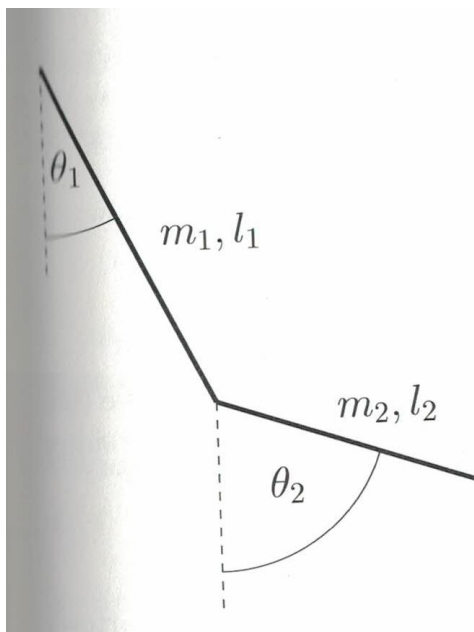
```
def pole(punkt,t=0):
    return (punkt[1],-np.sin(punkt[0]))
```

```
t=np.linspace(0,50,5001)
rozv=odeint(pole,[3,0],t)
plt.plot(t,rozv[:,0])
plt.plot(t,rozv[:,1])
plt.show()
```

Zad7 Wahadło podwójne składa się z dwóch jednorodnych prętów połączonych przegubowo, a koniec pierwszego z nich jest zamocowany trwale. Ruch układu odbywa się w ustalonej płaszczyźnie. Jego dynamika dana jest równaniami:

$$\begin{aligned} a\ddot{\theta}_1 + b \cos(\theta_1 - \theta_2)\ddot{\theta}_2 + b \sin(\theta_1 - \theta_2)\dot{\theta}_2^2 + p \sin \theta_1 &= 0 \\ c\ddot{\theta}_2 + b \cos(\theta_2 - \theta_1)\ddot{\theta}_1 + b \sin(\theta_2 - \theta_1)\dot{\theta}_1^2 + q \sin \theta_2 &= 0 \end{aligned}$$

gdzie $a = \left(\frac{1}{3}m_1 + m_2\right)l_1^2$, $c = \frac{1}{3}m_2l_2^2$, $b = \frac{1}{2}m_2l_1l_2$, $p = \left(\frac{1}{2}m_1 + m_2\right)gl_1$, $q = \frac{1}{2}m_2gl_2$.



Rozwiąż równanie dla punktu początkowego $[\frac{\pi}{2}, \frac{\pi}{2}, 0, 0]$ (układ odchylony do poziomu i puszczony z zerową prędkością). Wykonaj animację pierwszych 10 s rozwiązania z rysowaniem trajektorii wolnego końca wahadła dla parametrów fizycznych $m_1 = 1.5$ kg, $m_2 = 2$ kg, $l_1 = 1$ m, $l_2 = 0.6$ m.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint
from matplotlib.animation import FuncAnimation
```

```
m1=1.5; m2=2 # [kg]
```

```
l1=1; l2=0.6 # [m]
```

```
g=9.81 # [m/s^2]
```

```
a=m1*l1**2/3+m2*l1**2
```

```
c=m2*l2**2/3
```

```
b=m2*l1*l2/2
```

```
p=(m1/2+m2)*g*l1
```

```
q=m2*g*l2/2
```

```
def pole(X,t=0):
```

```

mianownik=a*c-(b*np.cos(X[1]-X[0]))**2
dteta1 = X[2]
dteta2 = X[3]
z1=b*np.sin(X[0]-X[1])*X[3]**2+p*np.sin(X[0])
z2=b*np.sin(X[1]-X[0])*X[2]**2+q*np.sin(X[1])
domega1= -(c*z1-b*np.cos(X[0]-X[1])*z2)/mianownik
domega2= -(a*z2-b*np.cos(X[1]-X[0])*z1)/mianownik
return [dteta1, dteta2, domega1, domega2]

```

```

t=np.linspace(0,10,1001)
rozw=odeint(pole,[3*np.pi/4,3*np.pi/4,0,0],t)

```

```

fig, ax = plt.subplots()
tx=[100*l1*np.sin(rozw[0,0])+100*l2*np.sin(rozw[0,1])]
ty=[-100*l1*np.cos(rozw[0,0])-100*l2*np.cos(rozw[0,1])]
tr=ax.plot(tx,ty,color='g')[0]

pret1 = ax.plot([0,100*l1*np.sin(rozw[0,0])], [0,-
100*l1*np.cos(rozw[0,0])],linewidth=2,color='r')[0]

pret2 = ax.plot([0,100*l1*np.sin(rozw[0,0])+100*l2*np.sin(rozw[0,1])], [0,-
100*l1*np.cos(rozw[0,0])-100*l2*np.cos(rozw[0,1])],linewidth=2,color='r')[0]

```

```

def anim(i):

    pret1.set_xdata([0,100*l1*np.sin(rozw[i,0])])
    pret1.set_ydata([0,-100*l1*np.cos(rozw[i,0])])

    pret2.set_xdata([100*l1*np.sin(rozw[i,0]),100*l1*np.sin(rozw[i,0])+100*l2*np.
sin(rozw[i,1])])

```

```
pret2.set_ydata([-100*l1*np.cos(rozw[i,0]),-100*l1*np.cos(rozw[i,0])-  
100*l2*np.cos(rozw[i,1]))]
```

```
tx.append(100*l1*np.sin(rozw[i,0])+100*l2*np.sin(rozw[i,1]))
```

```
ty.append(-100*l1*np.cos(rozw[i,0])-100*l2*np.cos(rozw[i,1]))
```

```
tr.set_xdata(tx)
```

```
tr.set_ydata(ty)
```

```
a = FuncAnimation(fig,anim,range(len(t)),interval=10,repeat=False)
```

```
ax.set_xlim([-240,240])
```

```
ax.set_ylim([-180,180])
```

```
plt.show()
```

Zad8-109

Ćwiczenie 109. Oblicz orbitę Ziemi wokół Słońca i wykonaj animację jej ruchu obiegowego. W animacji jedna sekunda powinna odpowiadać 30 dniom w rzeczywistości. Nieruchome Słońce oddziałuje na Ziemię siłą grawitacji:

$$\ddot{\vec{r}} = GM \frac{\vec{r}}{r^3}$$

gdzie $G = 6.674 \times 10^{-11} \frac{\text{N} \cdot \text{m}^2}{\text{kg}^2}$, a $M = 1.988 \times 10^{30} \text{ kg}$. W chwili początkowej Ziemia porusza się z prędkością $29.291 \frac{\text{km}}{\text{s}}$, prostopadłą do promienia o długości $1.52 \times 10^{11} \text{ m}$.

Rozwiązanie zad Zad8-109

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint
from matplotlib.animation import FuncAnimation
```

```
GM=6.674e-11*1.988e30*(8.64e4)**2*(1e-9)**3 # [Gm^3/d^2]
```

```
def pole(X,t=0):
    x,y,vx,vy=X
    dx=vx
    dy=vy
    m=(x**2+y**2)**1.5
    dvx=-GM*x/m
    dvy=-GM*y/m
    return [dx,dy,dvx,dvy]
```

```
wp=[152,0,0,2.53]
```

```
t=np.linspace(0,365,3651)
rozw=odeint(pole,wp,t)
rozw=rozw[:,6,:]
```

```
fig, ax = plt.subplots()
```

```
tx=[rozw[0,0]]
ty=[rozw[0,1]]
tr=ax.plot(tx,ty,color='g')[0]
z = ax.plot([rozw[0,0]],[rozw[0,1]],'ob')[0]
s = ax.plot([0],[0],'oy')[0]
```

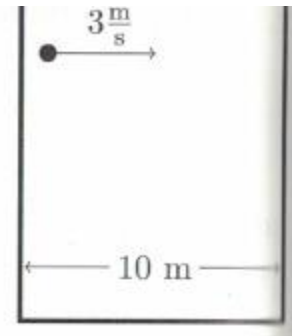
```
def anim(i):
    tx.append(rozw[i,0])
    ty.append(rozw[i,1])
    tr.set_xdata(tx)
    tr.set_ydata(ty)
    z.set_xdata(rozw[i,0])
    z.set_ydata(rozw[i,1])
```

```
a = FuncAnimation(fig,anim,range(len(rozw)),interval=20,repeat=False)
ax.set_xlim([-240,240])
ax.set_ylim([-180,180])
plt.show()
```

Zad9-110 Cząstka umieszczona jest w prostopadłościennym pudełku o szerokości 10m, poziomej podstawie

i prostopadłych ściankach. Działa na nią jedynie siła grawitacji $F = mg$ skierowana pionowo w dół. W chwili $t = 0$ znajduje się w punkcie $[1 \text{ m}, 10 \text{ m}]$ i porusza się poziomo z prędkością 3 m/s . Wykonaj animację pierwszych 15 s ruchu cząstki.

Wskazówka: Odbicie realizujemy jako siłę o bardzo dużej stałej wartości skierowaną poza obszarem do środka obszaru prostopadłe do jego brzegu.



Rozwiązanie Zad9-110

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint
from matplotlib.animation import FuncAnimation
```

```
def pole(p,t):
    x,y,vx,vy=p
    dx=vx
    dy=vy
    if x<0:
        dvx=1e6
    elif x>10:
        dvx=-1e6
    else:
        dvx=0
    if y<0:
        dvy=1e6
    else:
        dvy=-9.81
    return (dx,dy,dvx,dvy)
```

```

t=np.arange(0,15,.01)
rozw=odeint(pole,[1,10,3,0],t)

fig, ax = plt.subplots()
ax.plot([0,0,10,10],[15,0,0,15],linewidth=2)
k=ax.plot(rozw[:1,0],rozw[:1,1],'og')[0]
tr=ax.plot([],[])[0]

def anim(i):
    k.set_xdata([rozw[i,0]])
    k.set_ydata([rozw[i,1]])
    tr.set_xdata(rozw[:i,0])
    tr.set_ydata(rozw[:i,1])

a = FuncAnimation(fig,anim,range(len(t)),interval=10,repeat=False)
ax.set_xlim([-5.5,16.5])
ax.set_ylim([-0.5,16])
plt.show()

```